

Activity: Analyzing Amazon EC2 Instance Costs

The dataset contains 861 rows and 11 columns, which provide details about different Amazon EC2 instances. Here's a quick breakdown of the columns:

- **Name:** Name of the instance type.
- **API Name:** The API identifier for the instance.
- **Instance Memory:** Amount of memory (in GiB).
- **vCPUs:** Number of virtual CPUs.
- **Instance Storage:** Type of storage (e.g., EBS only).
- **Network Performance:** The network throughput level.
- **On Demand:** On-demand hourly cost for Linux instances.
- **Linux Reserved cost:** Reserved hourly cost for Linux instances.
- **Linux Spot Minimum cost:** Minimum spot price for Linux instances.
- **Windows On Demand cost:** On-demand hourly cost for Windows instances.
- **Windows Reserved cost:** Reserved hourly cost for Windows instances.

Part 1 : Analyze EC2 instance cost and performance data to understand pricing trends and identify potential cost outliers.

Step 1: Setup and Install Dependencies

Before starting, make sure you have the necessary libraries installed. If not, install them using `pip`.

```
pip install pandas matplotlib seaborn
```

Step 2: Load and Explore the Dataset

Import necessary libraries and load the CSV file into a Pandas DataFrame.

```
import pandas as pd

# Load the dataset
file_path = 'path_to_your_file/Amazon EC2 Instance Comparison.csv'
```

```
data = pd.read_csv(file_path)

# Display basic information about the dataset
print(data.info())

# Preview the first few rows
print(data.head())
```

Explanation: This code imports the `pandas` library to handle the data and loads the CSV into a `DataFrame`. The `.info()` and `.head()` methods help inspect the dataset.

Step 3: Clean the Data

Convert cost columns (which are stored as strings with '\$' symbols) into numeric data for analysis.

```
# List of cost-related columns
cost_columns = ['On Demand', 'Linux Reserved cost', 'Linux Spot Minimum cost', 'Windows On Demand cost', 'Windows Reserved cost']

# Remove '$' and 'hourly' and convert columns to numeric
for column in cost_columns:
    data[column] = pd.to_numeric(data[column].str.replace('[$, hourly]', '', regex=True), errors='coerce')

# Check for missing values after conversion
print(data[cost_columns].isnull().sum())
```

Step 4: Perform Summary Analysis

Generate summary statistics for cost columns to get an overview of the dataset.

```
# Generate summary statistics for cost-related columns
cost_summary = data[cost_columns].describe()
print(cost_summary)
```

Step 5: Visualize the Data

Use a boxplot to visualize the distribution of costs across different pricing options.

```
import matplotlib.pyplot as plt
import seaborn as sns

# Set up the plotting style
sns.set(style="whitegrid")
```

```
# Create a boxplot to visualize the distribution of costs
plt.figure(figsize=(12, 6))
sns.boxplot(data=data[cost_columns], palette="Set2")

# Set the plot labels and title
plt.title('Cost Comparison of Amazon EC2 Instances (Hourly)', fontsize=16)
plt.ylabel('Cost (USD)', fontsize=12)
plt.xticks(rotation=45, ha='right', fontsize=12)

# Display the plot
plt.tight_layout()
plt.show()
```

Step 6: Identify Outliers

Detect potential outliers in the cost data using interquartile ranges (IQR).

```
# Function to identify outliers using the IQR method
def detect_outliers(column):
    Q1 = data[column].quantile(0.25)
    Q3 = data[column].quantile(0.75)
    IQR = Q3 - Q1
    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR
    return data[(data[column] < lower_bound) | (data[column] > upper_bound)]

# Find outliers in the On-Demand cost column
outliers_on_demand = detect_outliers('On Demand')
print(outliers_on_demand)
```

Step 7: Conclusion and Further Analysis

Based on the analysis, explore further questions like:

- How do reserved instances compare to on-demand in terms of cost savings?
- What instance types are most cost-effective for specific needs (e.g., high performance at low cost)?

```
# Compare Reserved and On-Demand costs
cost_comparison = data[['Name', 'On Demand', 'Linux Reserved
cost']].dropna().sort_values('On Demand')
print(cost_comparison.head(10)) # Display the top 10 cost-effective
instances
```

Step 8: Filter and Compare Different Instance Families

Create a filter to isolate specific instance families (e.g., T2, T3, M5, etc.).

```
# Filter for specific instance families

def filter_instance_family(family):

    return data[data['Name'].str.startswith(family)]

# Example: Filter for T2 and T3 instance families

t2_instances = filter_instance_family('T2')

t3_instances = filter_instance_family('T3')

# Compare summary statistics for T2 and T3 instances

t2_summary = t2_instances[cost_columns].describe()

t3_summary = t3_instances[cost_columns].describe()

print("T2 Instance Costs Summary:\n", t2_summary)

print("\nT3 Instance Costs Summary:\n", t3_summary)
```

2. Visualize the cost distribution of different instance families.

```
# Visualize cost comparison for T2 and T3 instance families

plt.figure(figsize=(12, 6))

# Add T2 instances

sns.boxplot(data=t2_instances[cost_columns], palette="Blues", showmeans=True)

plt.title('Cost Distribution for T2 Instances', fontsize=16)

plt.ylabel('Cost (USD)', fontsize=12)

plt.xticks(rotation=45, ha='right', fontsize=12)

plt.show()
```

```
# Add T3 instances

plt.figure(figsize=(12, 6))

sns.boxplot(data=t3_instances[cost_columns], palette="Greens",
            showmeans=True)

plt.title('Cost Distribution for T3 Instances', fontsize=16)

plt.ylabel('Cost (USD)', fontsize=12)

plt.xticks(rotation=45, ha='right', fontsize=12)

plt.show()
```

3. Compare On-Demand and Reserved costs for different instance families.

```
# Compare On-Demand and Reserved costs for T2 and T3 families

comparison = pd.concat([t2_instances[['Name', 'On Demand', 'Linux Reserved
cost']],t3_instances[['Name', 'On Demand', 'Linux Reserved cost']]])

# Sort by On-Demand costs

comparison_sorted = comparison.dropna().sort_values('On Demand')

print(comparison_sorted.head(10)) # Display the 10 lowest-cost instances
```

Part 2: Predicting EC2 Instance Costs Using Regression Analysis

Objective:

Apply regression analysis to predict the cost of Amazon EC2 instances using Python. We'll use the `On-Demand` cost as the target variable and other instance features (like memory, vCPUs, etc.) as predictors.

Step 1: Setup and Install Dependencies

Make sure you have the necessary libraries for regression analysis.

```
pip install pandas scikit-learn matplotlib seaborn
```

Step 2: Load and Clean the Data

Load the dataset and clean the cost columns (as done in the previous steps).

```
import pandas as pd

# Load the dataset

file_path = 'path_to_your_file/Amazon EC2 Instance Comparison.csv'

data = pd.read_csv(file_path)

# Clean the cost columns by removing '$' and converting to numeric

cost_columns = ['On Demand', 'Linux Reserved cost', 'Linux Spot Minimum cost', 'Windows On Demand cost', 'Windows Reserved cost']

for column in cost_columns:

    data[column] = pd.to_numeric(data[column].str.replace('[$, hourly]', '', regex=True), errors='coerce')
```

Step 3: Feature Engineering

Convert relevant instance attributes into numerical format. We will use vCPUs and Instance Memory as predictor variables.

```
# Convert memory from string (e.g., '0.5 GiB') to numeric by stripping ' GiB'

data['Instance Memory'] = pd.to_numeric(data['Instance Memory'].str.replace(' GiB', ''))

# Extract the number of vCPUs from the 'vCPUs' column (e.g., '2 vCPUs')

data['vCPUs'] = pd.to_numeric(data['vCPUs'].str.extract('(\d+)', expand=False))

# Check the data after conversion

print(data[['Instance Memory', 'vCPUs']].head())
```

Step 4: Handle Missing Data

Handle any missing data by either removing or imputing missing values.

```
# Drop rows with missing values in the relevant columns (for simplicity)

data_cleaned = data.dropna(subset=['On Demand', 'Instance Memory', 'vCPUs'])

# Verify that the cleaned data has no missing values

print(data_cleaned.isnull().sum())
```

Step 5: Split the Data

Separate the dataset into features (predictors) and target (response) variables. In this case, we'll predict the On Demand cost.

```
from sklearn.model_selection import train_test_split

# Define features and target

X = data_cleaned[['Instance Memory', 'vCPUs']]

y = data_cleaned['On Demand']

# Split the data into training and testing sets (80% train, 20% test)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
                                                    random_state=42)

print(f"Training samples: {len(X_train)}, Testing samples: {len(X_test)}")
```

Step 6: Train a Linear Regression Model

Train a linear regression model using the training data.

```
from sklearn.linear_model import LinearRegression

# Create and train the model

model = LinearRegression()

model.fit(X_train, y_train)

# Print the coefficients of the model

print(f"Intercept: {model.intercept_}")
```

```
print(f"Coefficients: {model.coef_}")
```

- **Explanation:** The intercept and coefficients represent the relationship between the instance features (memory, vCPUs) and the On-Demand cost.

Step 7: Evaluate the Model

Evaluate the performance of the model using the test set.

```
from sklearn.metrics import mean_absolute_error, mean_squared_error

# Predict the costs for the test data
y_pred = model.predict(X_test)

# Calculate performance metrics
mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
rmse = mse ** 0.5

print(f"Mean Absolute Error (MAE): {mae}")
print(f"Mean Squared Error (MSE): {mse}")
print(f"Root Mean Squared Error (RMSE): {rmse}")
```

- **Explanation:** These metrics (MAE, MSE, RMSE) quantify how well the model predicts the costs. Lower values indicate better performance.

Step 8: Visualize the Results

Visualize the relationship between the actual and predicted costs using a scatter plot.

```
import matplotlib.pyplot as plt

# Plot actual vs predicted values
plt.figure(figsize=(8, 6))

plt.scatter(y_test, y_pred, alpha=0.7, color='b')
```

```
plt.plot([min(y_test), max(y_test)], [min(y_test), max(y_test)], color='red',
linestyle='--')

plt.title('Actual vs Predicted On-Demand Costs')

plt.xlabel('Actual On-Demand Cost')

plt.ylabel('Predicted On-Demand Cost')

plt.show()
```

- **Explanation:** The scatter plot shows how closely the predicted values align with the actual values. The red line represents a perfect prediction (actual = predicted).

Step 9: Make Predictions

You can now use the model to make predictions for new instance configurations (e.g., if you know the memory and vCPUs of a new instance).

```
# Example: Predict cost for a new instance with 4 GiB memory and 2 vCPUs

new_instance = [[4, 2]]

predicted_cost = model.predict(new_instance)

print(f"Predicted On-Demand Cost for 4 GiB, 2 vCPUs:
${predicted_cost[0]:.4f}")
```

- **Explanation:** This code predicts the cost for a hypothetical EC2 instance with 4 GiB of memory and 2 vCPUs using the trained model.